



Coordinating two agents in parallel-batching scheduling to minimize the number of late jobs

Renjie Yu, Daniel Oron^{ID*}

Discipline of Business Analytics, The University of Sydney Business School, Sydney, 2006, NSW, Australia

ARTICLE INFO

Keywords:

Single machine
Batching machine
Agent-scheduling
Metaheuristics

ABSTRACT

We study scheduling problems involving two competing agents sharing a single parallel-batching machine. The machine can process at most one job from each agent simultaneously as a batch. The processing time of a batch is equal to the largest processing time among the jobs in the batch. Each agent aims to minimize the number of their own late jobs. To account for the relative importance of the two agents, we consider two objective functions: minimizing the weighted sum of the objectives of both agents, and minimizing the objective of one agent subject to an upper bound on the other agent's objective. Both versions are shown to be strongly $\mathcal{NP}$ -hard. To address this complexity, we propose efficient Mixed-Integer Linear Programming (MILP) formulations. We then analyze the structural properties of optimal solutions, which motivate a constructive heuristic tailored to each problem. In addition, we develop two metaheuristic algorithms, Simulated Annealing and Tabu Search, incorporating neighborhood structures and search mechanisms designed to complement the limitations of the constructive heuristics. A comprehensive computational experiment validates the effectiveness and efficiency of the proposed algorithms. Finally, we explore a special case of the problem under consideration where the number of batches is limited to a constant. This case is shown to be solvable in polynomial time using a dynamic programming algorithm.

1. Introduction

In the current competitive environment, agents face significant challenges in reducing delivery times and lowering production costs. Machine scheduling plays a key role in achieving these goals. Classical machine scheduling assumes that only one job can be processed on a machine at any given time. However, in many repetitive manufacturing settings, machines can process multiple jobs simultaneously as a batch, and the processing time is determined by the longest job in the batch. This type of problem is known as parallel-batching machine scheduling (PBMS). Applications can be found across various contexts, such as aircraft engine manufacturing (Wang et al., 2017), tempered glass production (Liu et al., 2020), burn-in operations for integrated circuit chips (Ou et al., 2023), and autoclave molding processes for composite materials (Zheng et al., 2025). Meanwhile, two- or multi-agent scheduling is another important strand of research over the past two decades, initiated by Agnetis et al. (2004) and Baker and Cole Smith (2003). When multiple agents share the same machine resources, their interests frequently conflict as each aims to optimize their individual scheduling objectives.

In particular, our model design is motivated by the ship-loading process in mineral transportation. Consider a dual-arm shiploader responsible for loading two distinct types of minerals onto an arriving vessel. Each arm is connected to a separate conveyor belt, allowing the shiploader to load two minerals simultaneously as a batch, one from each conveyor. Since multiple orders must be processed for each mineral, the system naturally involves two agents, one for each mineral. The loading time of such a batch is determined by the slower stream, since both minerals must be transferred together until the longer order is completed. In practice, while the shiploader arms may be controlled independently, operational requirements such as balancing the vessel and meeting blending rules often enforce synchronized loading, which motivates this abstraction. Importantly, safety and quality-control constraints prohibit mixing two orders of the same mineral type, as this would cause cross-contamination. The shiploader must therefore coordinate between batching jobs from the two minerals to improve efficiency while ensuring that each agent's delivery deadlines are respected.

In this paper, we study two-agent scheduling problems where each agent has a disjoint set of jobs to be processed on a single parallel-batching machine. The jobs are characterized by different processing

* Corresponding author.

E-mail address: daniel.oron@sydney.edu.au (D. Oron).

times and due dates. The optimization goal of each agent is to minimize the number of its own late jobs. We assume that at most one job from each agent can be processed together in a batch, and the batch time is determined by the longer of the two jobs. We refer to this as the *agent-separation assumption*, which reflects practical constraints often seen in parallel-batching systems, such as energy usage, resource availability, and operational limits on thermal or pressure conditions. While agents compete for machine access, effective batching relies on cooperation. Specifically, batching jobs of similar length tends to reduce overall completion times and improve efficiency. This interplay between competition and cooperation motivates our research.

Beyond manufacturing and ship-loading, the proposed models can be applied to other scheduling scenarios. For instance, in cloud computing, tasks submitted by users compete for limited CPU and memory resources. Some tasks are CPU-intensive with low memory usage, while others are memory-intensive. These two categories can be viewed as two agents. Batching different types of tasks can improve resource utilization. However, tasks of the same type should not be batched together to avoid overloading specific resources. A similar structure may appear in healthcare logistics, where sterilization chambers may simultaneously accommodate instruments from different departments, but equipment from the same department cannot be processed together due to traceability requirements.

1.1. Literature review

This subsection reviews existing literature on PBMS and two-agent scheduling.

PBMS has attracted considerable research attention. A common way to classify PBMS problems is based on whether the batch capacity is limited. If not, the model is referred to as *unbounded*; otherwise, it is a *bounded* model. Our research focuses on the bounded model, as it is more relevant to practical applications. Another classification distinguishes between *compatible* and *incompatible* job settings. In such models, jobs are grouped into families, often with identical processing times within each family. In the incompatible setting, only jobs from the same family can be batched together. This family-based structure is conceptually similar to the agent-based setting, but the motivation differs. Job families are introduced to model compatibility constraints, whereas agents represent different stakeholders with individual interests and objectives. Recent studies have explored various variations of PBMS. For example, [Ou et al. \(2023\)](#) considered PBMS with release dates and job rejection, where the total rejection penalty must not exceed a given limit. With makespan as the objective, they derived structural properties and designed efficient approximation algorithms. [Yang et al. \(2022\)](#) studied PBMS with non-identical job sizes and incompatible job families, aiming to minimize the total weighted completion time. The authors proposed MILP formulations and a heuristic framework based on proximity search for problem solving. [Zheng et al. \(2025\)](#) investigated a series of PBMS problems targeting makespan minimization. They developed novel formulations, valid inequalities, and a solution framework that incorporates dual information and bounding techniques to improve efficiency. For a comprehensive review of recent advances in PBMS, we refer readers to the survey by [Fowler and Mönch \(2022\)](#). Notably, most existing PBMS studies assume that jobs are either independent or all belong to a single agent. The integration of parallel batching with multi-agent scheduling has been highlighted in the survey as a direction for future research.

There is limited research on two-agent scheduling problems involving a bounded parallel-batching machine. Below, we highlight several notable contributions. [Fan et al. \(2012\)](#) studied the problem of minimizing either the makespan or the total completion time of one agent, subject to an upper bound on the makespan of the other. The problem is shown to be solvable in polynomial time when agents are incompatible, meaning each batch includes jobs from only one agent. However, the problem becomes $\mathcal{NP}$ -hard when agents are compatible.

Building on this, [Wang et al. \(2017\)](#) considered the makespan minimization problem with non-identical job sizes but equal processing times. They proved the problem to be strongly $\mathcal{NP}$ -hard and developed two heuristics, reserved-space (RSH) and dynamic-mix (DMH), based on the first-fit decreasing (FFD) rule. Later, [Pei et al. \(2019\)](#) extended the constrained makespan model by incorporating an aging effect. To address this, they proposed a hybrid metaheuristic combining Bacterial Foraging Optimization with Variable Neighborhood Search. [Gao et al. \(2019\)](#) contributed another complexity result, proving that the problem remains $\mathcal{NP}$ -hard even under agreeable release dates and processing times, via a reduction from the Subset-Product Problem. [Zhang et al. \(2019\)](#) considered a different objective: minimizing the weighted sum of the agents' makespans. They provided a polynomial-time lower bound by relaxing the non-splittable job assumption and presented a constructive heuristic based on FFD.

It is important to note that all the above studies focus on non-due date-based objectives, namely the makespan and the total completion time. Due date-related objectives, by contrast, have received far less attention. [Brucker et al. \(1998\)](#) established that even the fundamental single-agent bounded PBMS with due dates is strongly $\mathcal{NP}$ -hard, using a reduction from the three-partition problem. This complexity barrier has limited further research in this direction, as it prevents exact algorithms for general cases. Some recent works have attempted to explore this challenging space under restrictive assumptions. For instance, [Feng et al. \(2020\)](#) studied a two-agent Pareto optimization problem involving both maximum lateness and makespan, assuming that all jobs have equal processing times. This assumption enabled the development of a polynomial-time algorithm to enumerate all Pareto-optimal solutions. [Tang et al. \(2017\)](#) investigated a constrained problem with linearly deteriorating jobs, targeting the number of late jobs. Their results included complexity and algorithmic insights under conditions such as fixed machine capacity and agreeability among release dates, due dates, and processing times. However, some of their conclusions were later revised by [Kovalyov and Šešok \(2019\)](#), who showed that the considered problems are in fact strongly $\mathcal{NP}$ -hard, correcting the earlier classification as ordinary $\mathcal{NP}$ -hard.

To the best of our knowledge, existing research on two-agent parallel-batching scheduling either avoids due date-based objectives or relies on simplifying assumptions to reduce complexity. However, as seen in many of the earlier applications, batch composition is often limited in practice. This motivates the adoption of the agent-separation assumption in our model.

1.2. Statement of contribution and paper organization

We briefly summarize our key contributions to the literature as follows:

1. This is one of the very few papers to study two-agent parallel-batching scheduling with due dates. We demonstrate the real-life relevance of limited batching capacity and the agent-separation assumption.
2. We prove the strong $\mathcal{NP}$ -hardness of the proposed problems and analyze their structural properties, which guide the design of constructive heuristics and neighborhood moves for effective search.
3. We propose a compact MILP formulation and develop a polynomial-time dynamic programming-based algorithm for a parameterized special case, demonstrating tractability in restricted settings.
4. We design coherent neighborhood-based metaheuristics, namely Simulated Annealing (SA) and Tabu Search (TS). The methods are carefully tailored to the problem structure and bottlenecks, with special features introduced and tested.
5. We conduct extensive computational experiments to verify both the effectiveness and efficiency of the proposed approaches, and we provide practical insights into their relative strengths.

The remainder of the paper is organized as follows. Section 2 formally defines the problem, along with the adopted assumptions and notations, and presents the MILP formulations. Section 3 establishes the computational complexity of the proposed problems. Section 4 examines the structural properties of optimal schedules. Based on these properties, we propose constructive heuristics, which are used as initial solutions for the metaheuristic algorithms developed in Section 5. Section 6 presents computational experiments evaluating the performance of the proposed algorithms. Section 7 explores a case where the number of batches is limited to a constant. Finally, Section 8 concludes the paper and discusses directions for future research.

2. Problem statement

2.1. Assumptions and notations

There are two competing agents, Agent A and Agent B , each processing a distinct set of jobs on a single parallel-batching machine without preemption. For $k \in \{A, B\}$, each agent k has a set of jobs denoted by $\mathcal{J}^k = \{J_1^k, J_2^k, \dots, J_{n_k}^k\}$. For convenience, we refer to each job J_i^A of Agent A as an A -job, and each job J_j^B of Agent B as a B -job, for $i = 1, \dots, n_A$ and $j = 1, \dots, n_B$. The processing time and due date of each job J_i^A (or J_j^B) are denoted by p_i^A (or p_j^B) and d_i^A (or d_j^B), respectively. All jobs are available for processing from time zero, i.e., the release dates of all jobs are $r_i^A = r_j^B = 0$. The machine can either process a single job individually or simultaneously process one A -job and one B -job as a batch. For any batch containing jobs J_i^A and J_j^B , denoted by $B = \{J_i^A, J_j^B\}$, its processing time is $p(B) = \max\{p_i^A, p_j^B\}$.

For any schedule π , the completion times of jobs J_i^A and J_j^B are denoted by C_i^A and C_j^B , respectively. Let U_i^A and U_j^B be binary indicators, where $U_i^A = 1$ if $C_i^A > d_i^A$, and $U_j^B = 1$ if $C_j^B > d_j^B$. Otherwise, $U_i^A = 0$ and $U_j^B = 0$. Without loss of generality, we assume $d_i^A \geq p_i^A + r_i^A$ and $d_j^B \geq p_j^B + r_j^B$ for all $i = 1, \dots, n_A$ and $j = 1, \dots, n_B$; otherwise, the job must be late and can be eliminated from consideration. Both agents share the same objective function to minimize the number of late jobs.

To balance the objectives of the two agents and model their relative priority, we employ two bi-criteria optimization forms. The first form minimizes a weighted sum of the number of late jobs from both agents. Each A -job is assigned a weight α , while each B -job is assigned a weight $1 - \alpha$, where $0 < \alpha < 1$. This form allows adjusting the importance of each agent's objective continuously. The second form optimizes the objective value of Agent A while placing an upper bound Q on that of Agent B . This form reflects cases where Agent B must meet a strict performance limit, while Agent A optimizes its own performance under that constraint.

Following the standard three-field notations $\alpha | \beta | \gamma$ introduced by Graham et al. (1979), we denote our problems as $1 | p\text{-batch}; b = 2 | \alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$ and $1 | p\text{-batch}; b = 2 | U_i^A : U_j^B \leq Q$, where $p\text{-batch}; b = 2$ indicates that the parallel-batching machine can process up to two jobs as a batch.

For illustration, we provide the following numerical examples to show how jobs are batched and how lateness is calculated.

- *Instance 1.* Each agent has two jobs. The processing times are $p_1^A = 1, p_2^A = 4, p_1^B = 3, p_2^B = 2$. The due dates are $d_1^A = 2, d_2^A = 6, d_1^B = 5, d_2^B = 6$.
- *Instance 2.* Agent A has two jobs, and Agent B has one job. The processing times are $p_1^A = 1, p_2^A = 3, p_1^B = 4$. The due dates are $d_1^A = 5, d_2^A = 6, d_1^B = 4$.

Figs. 1 and 2 illustrate the solutions for these instances. For each instance, the left-hand schedule is suboptimal, with a late job marked in red. In *Instance 1*, J_2^B is late since $C_2^B = p_1^A + \max\{p_2^A, p_1^B\} + p_2^B = 7 > d_2^B = 6$. In *Instance 2*, J_2^A is late since $C_2^A = \max\{p_1^A, p_1^B\} + p_2^A = 7 > d_2^A = 6$.

2.2. Mathematical formulation

Given that there are $n = n_A + n_B$ jobs in total, the machine can process all jobs within at most n rounds if no batching occurs. We provide a position-based MILP with n positions for the problem $1 | p\text{-batch}; b = 2 | \alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$ as follows.

$$\min \alpha \sum_{i=1}^{n_A} U_i^A + (1 - \alpha) \sum_{j=1}^{n_B} U_j^B \quad (1)$$

$$\text{s.t.} \quad \sum_{i=1}^{n_A} x_{ik} \leq 1, \quad k = 1, \dots, n \quad (2)$$

$$\sum_{j=1}^{n_B} y_{jk} \leq 1, \quad k = 1, \dots, n \quad (3)$$

$$\sum_{k=1}^n x_{ik} = 1, \quad i = 1, \dots, n_A \quad (4)$$

$$\sum_{k=1}^n y_{jk} = 1, \quad j = 1, \dots, n_B \quad (5)$$

$$u_k \geq x_{ik}, \quad i = 1, \dots, n_A, \quad k = 1, \dots, n \quad (6)$$

$$u_k \geq y_{jk}, \quad j = 1, \dots, n_B, \quad k = 1, \dots, n \quad (7)$$

$$u_k \geq u_{k+1}, \quad k = 1, \dots, n \quad (8)$$

$$P_k \geq x_{ik} p_i^A, \quad i = 1, \dots, n_A, \quad k = 1, \dots, n \quad (9)$$

$$P_k \geq y_{jk} p_j^B, \quad j = 1, \dots, n_B, \quad k = 1, \dots, n \quad (10)$$

$$C_1 \geq P_1 \quad (11)$$

$$C_k \geq P_k + C_{k-1}, \quad k = 2, \dots, n \quad (12)$$

$$C_k - d_i^A - M^*(1 - x_{ik}) \leq M^* U_i^A, \quad i = 1, \dots, n_A, \quad k = 1, \dots, n \quad (13)$$

$$C_k - d_j^B - M^*(1 - y_{jk}) \leq M^* U_j^B, \quad j = 1, \dots, n_B, \quad k = 1, \dots, n \quad (14)$$

$$U_i^A, U_j^B, x_{ik}, y_{jk}, u_k \in \{0, 1\} \quad i = 1, \dots, n_A, \quad j = 1, \dots, n_B, \quad k = 1, \dots, n \quad (15)$$

$$P_k, C_k \geq 0, \quad i = 1, \dots, n_A, \quad j = 1, \dots, n_B, \quad k = 1, \dots, n \quad (16)$$

where

- x_{ik} and y_{jk} are binary decision variables. Specifically, $x_{ik} = 1$ ($y_{jk} = 1$) if job J_i^A (J_j^B) is completed at position k of the sequence, and $x_{ik} = 0$ ($y_{jk} = 0$) otherwise.
- u_k is a binary variable equal to 1 if at least one job occupies position k , 0 otherwise.
- P_k and C_k are nonnegative variables representing the processing time and the completion time of the batch (or single job) allocated to position k , respectively.
- U_i^A and U_j^B are binary variables indicating whether jobs J_i^A and J_j^B are late or not, respectively.
- M^* is pre-computed as $\sum_{i=1}^{n_A} p_i^A + \sum_{j=1}^{n_B} p_j^B - \min\{\min_{i=1, \dots, n_A} \{d_i^A\}, \min_{j=1, \dots, n_B} \{d_j^B\}\}$, corresponding to the maximum possible lateness among all jobs.

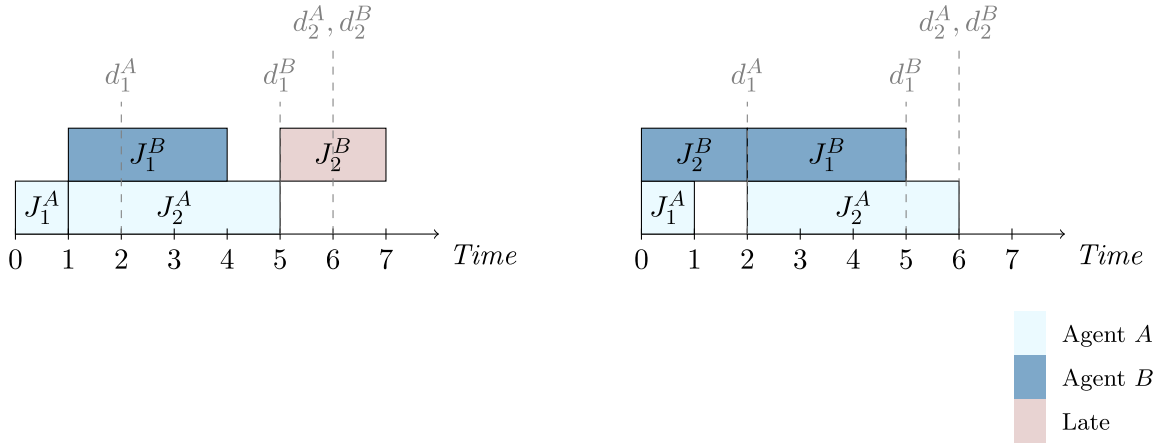


Fig. 1. Numerical example *Instance 1*. Left: the initial solution. Right: the solution refined by move *BatchLate*.

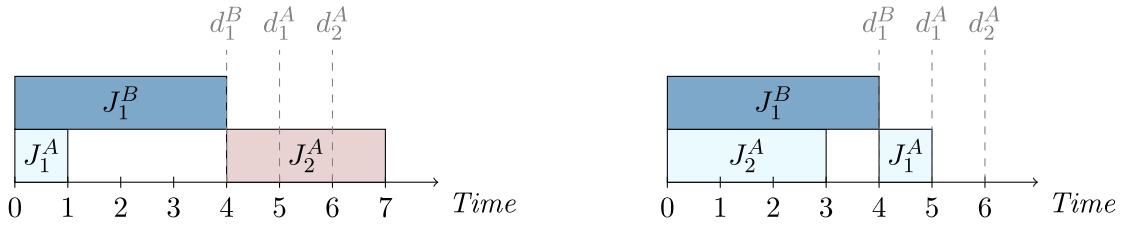


Fig. 2. Numerical example *Instance 2*. Left: the initial solution. Right: the solution refined by move *SwapBatch*.

The objective (1) minimizes the total weighted number of late jobs for both agents. Constraints (2) and (3) ensure at most one job per agent occupies each position. Constraints (4) and (5) guarantee each job is scheduled exactly once. Constraints (6) and (7) link position utilization variables to job assignments, while constraint (8) eliminates scheduling symmetry by enforcing that empty positions, if any, appear only at the end of the schedule. Constraints (9)–(12) determine the processing time and the completion time of the batch (or single job) in each position. Constraints (13) and (14) use a standard big- M formulation to linearize the logical condition that a job is late when its completion time exceeds its due date.

For the constrained problem 1 | p -batch; $b = 2$ | $U_i^A : U_j^B \leq Q$, we modify the above MILP formulation by replacing the objective function with objective (17) and adding constraint (18), which enforces the upper bound Q on the number of late jobs for Agent B .

$$\min \sum_{i=1}^{n_A} U_i^A \quad (17)$$

$$\sum_{j=1}^{n_B} U_j^B \leq Q \quad (18)$$

3. Proof of $\mathcal{NP}$ -hardness

Brucker et al. (1998) established that the single-agent problem 1 | p -batch; $b = 2$ | $\sum U_j$ is strongly $\mathcal{NP}$ -hard. Note that our proposed two-agent problems are not direct generalizations of the single-agent case, as the additional agent-separation constraint alters the feasible set in a non-trivial way. In the single-agent problem, any two jobs can be combined into a batch, whereas in our setting each batch must contain at most one job from each agent. This restriction removes many feasible schedules that would otherwise be possible in the single-agent model, so the complexity of the two-agent problems cannot be inferred directly from the single-agent case. Below, we adapt their reduction from the 3-PARTITION problem to show that the problem 1 | p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$ remains strongly $\mathcal{NP}$ -hard, even when both agents are equally weighted, i.e., $\alpha = 0.5$.

Table 1

Jobs created in the reduction from 3-PARTITION.

Job	p	d
$J_{1,0}^B$	$p_{1,0} = 1 \cdot W + t a_1$	$d_t = \sum_{\ell=1}^t \left(\sum_{\ell=1}^{3t} \ell W + (t-h)tA + hA \right)$
$J_{1,1}^A$	$p_{1,1} = 1 \cdot W + (t-1)a_1$	$d_1 = \left(\sum_{\ell=1}^{3t} \ell W + (t-1)tA + hA \right)$
$J_{1,2}^A, J_{1,2}^B$	$p_{1,2} = 1 \cdot W + (t-2)a_1$	$d_2 = \sum_{h=1}^2 \left(\sum_{\ell=1}^{3t} \ell W + (t-h)tA + hA \right)$
$\vdots$	$\vdots$	$\vdots$
$J_{1,t}^A, J_{1,t}^B$	$p_{1,t} = 1 \cdot W$	$d_t = \sum_{h=1}^t \left(\sum_{\ell=1}^{3t} \ell W + (t-h)tA + hA \right)$
$\vdots$	$\vdots$	$\vdots$
$J_{3t,0}^B$	$p_{3t,0} = 3t \cdot W + t a_{3t}$	$d_t = \sum_{h=1}^t \left(\sum_{\ell=1}^{3t} \ell W + (t-h)tA + hA \right)$
$J_{3t,1}^A$	$p_{3t,1} = 3t \cdot W + (t-1)a_{3t}$	$d_1 = \left(\sum_{\ell=1}^{3t} \ell W + (t-1)tA + hA \right)$
$J_{3t,2}^A, J_{3t,2}^B$	$p_{3t,2} = 3t \cdot W + (t-2)a_{3t}$	$d_2 = \sum_{h=1}^2 \left(\sum_{\ell=1}^{3t} \ell W + (t-h)tA + hA \right)$
$\vdots$	$\vdots$	$\vdots$
$J_{3t,t}^A, J_{3t,t}^B$	$p_{3t,t} = 3t \cdot W$	$d_t = \sum_{h=1}^t \left(\sum_{\ell=1}^{3t} \ell W + (t-h)tA + hA \right)$

Definition 1 (3-PARTITION). Given an integer A , a finite set of $3t$ positive integers, denoted by $H = \{a_1, \dots, a_{3t}\}$, such that $\frac{A}{4} < a_\ell < \frac{A}{2}$ for $\ell = 1, \dots, 3t$, and $\sum_{\ell=1}^{3t} a_\ell = tA$, can the index set $\{1, \dots, 3t\}$ be partitioned into t disjoint subsets $H_1, \dots, H_t$ such that $\sum_{\ell \in H_r} a_\ell = A$, for $r = 1, \dots, t$?

Theorem 3.1. The problem 1 | p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$ is strongly $\mathcal{NP}$ -hard, even when $\alpha = 0.5$.

Proof. Given an arbitrary 3-PARTITION instance with $3t$ integers ($H = \{a_1, \dots, a_{3t}\}$) and target A , we construct the following instance for the scheduling problem 1 | p -batch; $b = 2$ | $\sum U_i^A + \sum U_j^B$ (corresponding to the special case $\alpha = 0.5$), where each agent has $3t^2$ jobs.

For Agent A , let the jobs be denoted $J_{\ell,r}^A$ for $\ell = 1, \dots, 3t$ and $r = 1, \dots, t$. For Agent B , the jobs are divided into two sets: (i) $J_{\ell,0}^B$,

and (ii) $J_{\ell,r}^B$ for $r = 2, \dots, t$, for each $\ell = 1, \dots, 3t$. All jobs of the same index ℓ are referred to as being of type ℓ .

The processing times and due dates are defined as follows:

$$p_{\ell,r} = \ell W + (t-r)a_{\ell}, \quad d_r = \sum_{h=1}^r \left(\sum_{\ell=1}^{3t} \ell W + (t-h)tA + hA \right),$$

where $W = t^3 A$ and $d_0 = d_t$. Note that for a fixed ℓ , the processing time $p_{\ell,r}$ is strictly decreasing in r . Table 1 summarizes the jobs created in the reduction.

We maintain the constraint that each batch must contain exactly one job from each agent. We now show that there exists a schedule with no late jobs (i.e., $\sum U_i^A + \sum U_j^B = 0$) if and only if the 3-PARTITION instance has a solution.

If Part. Suppose the 3-PARTITION instance admits a solution $\langle H_1, \dots, H_t \rangle$. We construct a schedule divided into t blocks. For $r = 1, \dots, t$, each block r consists of $3t$ batches:

- The first 3 batches are $\{J_{\ell,r}^A, J_{\ell,0}^B\}$ for each $\ell \in H_r$.
- The remaining $3t-3$ batches are $\{J_{\ell,r}^A, J_{\ell,r}^B\}$ if $r \neq 1$, or $\{J_{\ell,r}^A, J_{\ell,r+1}^B\}$ if $r = 1$, where $\ell \notin H_r$.

In the latter batches, the job from Agent A always has a larger processing time. The completion time of block r is given by:

$$C_r = \sum_{h=1}^r \left(\sum_{\ell=1}^{3t} \ell W + \sum_{\ell \in H_h} t a_{\ell} + \sum_{\ell \notin H_h} (t-h)a_{\ell} \right) = d_r.$$

Thus, all jobs in each block are completed no later than their due dates.

Only if Part. Conversely, suppose there exists a feasible schedule π such that all jobs are completed on time. While jobs from the same agent cannot be batched, we first claim that each batch must consist of two jobs of the same type. Otherwise, the completion time of the last job exceeds the latest due date:

$$C_{\max} > \sum_{h=1}^t \sum_{\ell=1}^{3t} \ell W + W \geq \sum_{h=1}^t \left(\sum_{\ell=1}^{3t} \ell W + t^2 A \right) \geq d_t,$$

which contradicts feasibility.

We next show by induction that each job $J_{\ell,r}^A$ of Agent A must be scheduled in block r , specifically in positions $3(r-1)t+1$ to $3rt$. For $r = 1$, to ensure all $J_{\ell,1}^A$ finish before d_1 , they must be scheduled in the first $3t$ positions, as $\sum_{\ell=1}^{3t} p_{\ell,1} > \sum_{\ell=1}^{3t} \ell W \geq d_1 - W$. A similar argument consecutively applies for each $r = 2, \dots, t$. Since each batch consists of jobs of the same type, each batch containing $J_{\ell,r}^A$ has processing time $p_{\ell,r}$, unless it is paired with $J_{\ell,0}^B$, in which case it requires $p_{\ell,0}$. Let $\ell \in H_r$ if batch $\{J_{\ell,r}^A, J_{\ell,0}^B\}$ exists in the given schedule. Then $\langle H_1, \dots, H_t \rangle$ defines a 3-PARTITION. As all jobs in each block r are early, it must also hold that

$$C_r = \sum_{h=1}^r \left(\sum_{\ell \in H_h} p_{\ell,0} + \sum_{\ell \notin H_h} p_{\ell,h} \right) \leq d_r, \quad r = 1, \dots, t,$$

which implies

$$\sum_{h=1}^r h(A - \sum_{\ell \in H_h} a_{\ell}) \geq 0, \quad r = 1, \dots, t.$$

Assume one inequality is strict. Then:

$$\sum_{h=1}^t \left(A - \sum_{\ell \in H_h} a_{\ell} \right) \left(\sum_{r=h}^{t-1} \left(\frac{1}{r} - \frac{1}{r+1} \right) + \frac{1}{t} \right) > 0,$$

which simplifies to:

$$tA - \sum_{h=1}^t \sum_{\ell \in H_h} a_{\ell} > 0,$$

contradicting the condition $\sum_{\ell=1}^{3t} a_{\ell} = tA$. Thus, $\sum_{\ell \in H_r} a_{\ell} = A$ for all r , and $\langle H_1, \dots, H_t \rangle$ is a valid solution to the 3-PARTITION instance. $\square$

For clarity, we provide a high-level summary of the reduction logic. The key idea is to establish a two-way equivalence between the two problems. The t subsets in a 3-PARTITION correspond to t blocks in the schedule, where each block contains exactly $3t$ batches. Within each block, three special batches are reserved for the three numbers assigned to that group, while the remaining batches handle the other jobs. The due dates d_r are designed so that block r must complete exactly on time if and only if the three chosen numbers in that block sum to A . Thus, the structure of the schedule directly encodes a feasible partition.

For the constrained problem $1 \mid p\text{-batch}; b = 2 \mid U_i^A : U_j^B \leq Q$, the same reduction applies by setting $Q = 0$ and maintaining that all jobs from both agents must be completed early. Therefore, we obtain the following corollary.

Corollary 3.1.1. *The problem $1 \mid p\text{-batch}; b = 2 \mid U_i^A : U_j^B \leq Q$ is strongly $\mathcal{NP}$ -hard.*

The complexity results imply that, unless $\mathcal{P} = \mathcal{NP}$, no algorithm can produce an optimal schedule for either problem in polynomial or pseudo-polynomial time. As shown in the computational experiments in Section 6, Gurobi solver fails to solve the proposed MILP models for some instances with more than 10 jobs per agent within a reasonable time limit (1800s). This motivates the development of heuristic algorithms to improve computational efficiency.

4. Structural properties and constructive heuristics

In this section, we present some structural results that are useful for developing heuristic solution methods for the proposed problems.

4.1. Structural properties

Lemma 4.1. *There exists an optimal schedule in which all early jobs processed individually, regardless of their agent, are scheduled in nondecreasing order of their due dates, i.e., the earliest due date first (EDD) rule.*

Proof. Scheduling early jobs without batching reduces to the classical two-agent late job minimization problem. Thus, this property follows from a simple pairwise interchange argument. $\square$

Lemma 4.2. *There exists an optimal schedule in which all batched jobs are early, and all late jobs are scheduled arbitrarily at the end, after all early jobs have been completed.*

Proof. Suppose there exists an optimal schedule in which some late jobs are scheduled before early jobs. Consider moving each such late job to the end of the schedule. The early jobs shift backward while still satisfying their due dates, thereby remaining early. Then consider a batch that contains both an early and a late job. We split the batch and delay the late job to the end. As the batch length is determined by the longer of the two jobs, removing a late job can only reduce or maintain the completion times of the remaining jobs. Thus, the objective value will not worsen. $\square$

As expected, the EDD rule remains effective in most cases, consistent with classical two-agent late job minimization problems. However, when batch processing is allowed, the interaction between batched and unbatched jobs introduces additional complexity that does not arise in the classical setting.

For illustration, we again look at the previous numerical example in Section 2. Figs. 1 and 2 illustrate the solutions for the two instances. The left-hand schedule follows the EDD rule, while the right-hand schedule presents the only optimal solution. It can be observed that, in the optimal solution of Instance 1, both jobs of Agent B are batched, but do not follow the EDD rule. In Instance 2, the two jobs of Agent A (one is batched, one is not) do not follow either the EDD rule or the shortest processing time first (SPT) rule.

Remark. There are several common special cases (assumptions) considered in the literature mentioned in Section 1.1.

- Case (i): When a common due date is assumed for all jobs or for all jobs of each agent.
- Case (ii): When the jobs of each agent have the same processing time.
- Case (iii): When the processing times and due dates of all jobs are globally agreeable or reversely agreeable.

In these cases, the structural relationship is straightforward and follows from standard pairwise interchange arguments, in addition to the two general properties established above. For Case (i), there exists an optimal solution where the jobs of each agent follow the SPT rule. For Cases (ii) and (iii), there exists an optimal solution where the jobs of each agent follow the EDD rule, regardless of whether they are batched or not.

4.2. Heuristic for problem 1 | p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$

We develop the following constructive heuristic for problem 1 | p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$. The heuristic prioritizes individually processed jobs and sequences them according to the EDD rule, as indicated by Lemma 4.1. The metaheuristics discussed in subsequent sections will refine this initial solution by exploring the interaction among batched jobs and seeking further improvements.

Without loss of generality, we assume that all jobs from Agents A and B are indexed according to their EDD order, respectively. Following Lemma 4.2, we assume that each schedule, denoted by (π^E, π^L) , consists of an early job set π^E scheduled at the beginning, followed by a late job set π^L . The heuristic operates as a forward greedy procedure, as described in Algorithm (1).

At each iteration, the heuristic considers the next available pair of EDD-ordered jobs, one from each agent if available, and continues until all jobs are scheduled. The algorithm evaluates three options in sequence. First, it checks whether the current job can be batched with the job scheduled individually in the previous iteration. If this batch can be completed early, it replaces the previous singleton with the batch and moves to the next iteration. If this is not possible, the heuristic then checks whether the current pair of jobs can be batched together and still finish early. If so, the batch is scheduled. If neither batching option is feasible, the algorithm schedules one of the current jobs individually while prioritizing the one that can still meet its due date. If neither job can be completed on time, the job (or both jobs) is assigned to the late job set, following Lemma 4.2.

Our motivation is to maximize batching opportunities while ensuring that all batched jobs are completed early. Batching helps reduce overall completion time and creates space in the schedule for more early jobs.

4.3. Heuristic for problem 1 | p -batch; $b = 2$ | $U_i^A : U_j^B \leq Q$

Clearly, both objective forms share a similar solution structure. However, for the constrained problem, we must account for the upper bound Q imposed on the number of late jobs from Agent B .

The heuristic begins by running Algorithm (1) and computing the number of late B -jobs in the resulting schedule, denoted by f_0^B . This leads to the following two cases:

- **Case (i):** $f_0^B \leq Q$. The output solution is feasible for the constrained problem and is directly used as the initial solution.

Algorithm 1: A constructive heuristic for 1 | p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$

Input : Job sets $\mathcal{J}^A, \mathcal{J}^B; \alpha$.
Output : An approximate solution with early job set π^E and late job set π^L .
Initialization: $i = j = T = 0$, $batchDone = False$,
 $lastJob = None$, $\pi^E = \pi^L = \emptyset$.

- Sort all jobs of $\mathcal{J}^A$ and $\mathcal{J}^B$ according to the EDD rule, respectively.
- while** $i \leq n_A$ **or** $j \leq n_B$ **do**
- $batchDone \leftarrow False$;
- if** $lastJob = A$ **and** $j < n_B$ **and**
 $T - p_{i-1}^A + \max\{p_{i-1}^A, p_j^B\} \leq \min\{d_{i-1}^A, d_j^B\}$ **then**
- Remove J_{i-1}^A from π^E , add $\{J_{i-1}^A, J_j^B\}$ to π^E ;
- $T \leftarrow T - p_{i-1}^A + \max\{p_{i-1}^A, p_j^B\}$, $j \leftarrow j + 1$,
 $batchDone \leftarrow True$;
- end**
- else if** $lastJob = B$ **and** $i < n_A$ **and**
 $T - p_{j-1}^B + \max\{p_i^A, p_{j-1}^B\} \leq \min\{d_i^A, d_{j-1}^B\}$ **then**
- Remove J_{j-1}^B from π^E , add $\{J_i^A, J_{j-1}^B\}$ to π^E ;
- $T \leftarrow T - p_{j-1}^B + \max\{p_i^A, p_{j-1}^B\}$, $i \leftarrow i + 1$,
 $batchDone \leftarrow True$;
- end**
- if** $batchDone$ **then**
- $lastJob \leftarrow None$;
- continue**;
- end**
- if** $i < n_A$ **and** $j < n_B$ **and** $T + \max\{p_i^A, p_j^B\} \leq \min\{d_i^A, d_j^B\}$ **then**
- Add $\{J_i^A, J_j^B\}$ to π^E ;
- $T \leftarrow T + \max\{p_i^A, p_j^B\}$, $i \leftarrow i + 1$, $j \leftarrow j + 1$,
 $lastJob \leftarrow None$;
- continue**;
- end**
- if** $i < n_A$ **then**
- if** $T + p_i^A \leq d_i^A$ **then**
- Add J_i^A to π^E , $T \leftarrow T + p_i^A$, $i \leftarrow i + 1$, $lastJob \leftarrow A$;
- continue**;
- end**
- else**
- Add J_i^A to π^L , $i \leftarrow i + 1$;
- end**
- end**
- if** $j < n_B$ **then**
- if** $T + p_j^B \leq d_j^B$ **then**
- Add J_j^B to π^E , $T \leftarrow T + p_j^B$, $j \leftarrow j + 1$, $lastJob \leftarrow B$;
- end**
- else**
- Add J_j^B to π^L , $j \leftarrow j + 1$;
- end**
- end**
- end**

- **Case (ii):** $f_0^B > Q$. The solution violates the constraint on late B -jobs and is thus infeasible. To resolve this, we apply an iterative repair procedure described below.

We iteratively remove one A -job at a time and re-run Algorithm 1 on the reduced job set. Since Agent A has no constraint on the number of late jobs, the removed A -jobs are assigned to the late job set, thereby freeing up capacity in the early part of the schedule and

potentially allowing more B -jobs to be scheduled early. Clearly, the extreme case occurs when all A -jobs are considered late and only B -jobs are scheduled, which guarantees a feasible solution as long as the overall problem is feasible.

Our experimental results indicate that prioritizing the removal of A -jobs with the longest processing times is most effective, as such jobs tend to consume more time and limit scheduling flexibility. The process continues until a schedule is found where the number of late B -jobs does not exceed Q .

The entire procedure is summarized in Algorithm (2), and the final output is a feasible schedule that serves as the initial solution for metaheuristics.

Algorithm 2: A heuristic for $1 \mid p\text{-batch}; b = 2 \mid U_i^A : U_j^B \leq Q$

Input : Job sets $\mathcal{J}^A, \mathcal{J}^B$; upper bound Q
Output : Feasible schedule (π^E, π^L) such that $U_j^B \leq Q$
Initialization: $\mathcal{J}_A^{\text{remain}} \leftarrow \mathcal{J}^A, \mathcal{J}_A^{\text{late}} \leftarrow \emptyset$

```

1 while true do
2   Run Algorithm 1 on  $(\mathcal{J}_A^{\text{remain}}, \mathcal{J}^B)$ ;
3   Let  $(\pi^E, \pi^L)$  be the resulting early and late job sets;
4   Let  $f^B \leftarrow$  number of  $B$ -jobs in  $\pi^L$ ;
5   if  $f^B \leq Q$  then
6     break;
7   end
8   Select  $J_i^A \in \mathcal{J}_A^{\text{remain}}$  with the largest processing time  $p_i^A$ ;
9    $\mathcal{J}_A^{\text{remain}} \leftarrow \mathcal{J}_A^{\text{remain}} \setminus \{J_i^A\}$ ;
10   $\mathcal{J}_A^{\text{late}} \leftarrow \mathcal{J}_A^{\text{late}} \cup \{J_i^A\}$ ;
11 end
12 Append all jobs in  $\mathcal{J}_A^{\text{late}}$  to  $\pi^L$ ;
```

5. Proposed metaheuristic algorithms

In this section, we present two metaheuristic algorithms, Tabu Search (TS) and Simulated Annealing (SA), to obtain approximate solutions for the proposed problems. These two methods have been widely used and demonstrated effective performance in various scheduling contexts. Some recent applications of SA can be found in Beldar et al. (2022), Fontes et al. (2023), and Shafiee et al. (2025), while TS has been applied in works such as Chen et al. (2022) and Shahvari and Legendran (2017).

Although both algorithms employ the same neighborhood moves, their search dynamics differ in important ways. SA explores the solution space stochastically, accepting worse solutions with a probability that decreases as the temperature cools. This mechanism provides robustness in highly irregular search spaces. In our setting, the agent-separation rule and lateness-based objective create such ruggedness. Neighborhood moves are often restricted, and small changes in batching decisions can cause large shifts in lateness. This feature aligns well with the structure of our problem. TS, in contrast, always chooses the best admissible neighbor. It is more structured and excels in intensification, systematically avoiding cycling through its tabu list. The following subsections describe the detailed design of each algorithm.

5.1. Neighborhood structures

We use the constructive heuristics (Algorithms (1) and (2)) to generate initial solutions for the two proposed problems. Recall that these heuristics primarily focus on scheduling unbatched jobs and forming as many batches as possible, leaving room for improvement in the interaction between batched jobs. To address this limitation, we propose the following neighborhood structures, which are employed by the metaheuristic algorithms to improve the solutions:

- **InsertLate:** For each job currently in the late job set π^L , attempt to insert it into the early job set π^E at an appropriate position.
- **BatchLate:** For each late job in the late job set π^L , attempt to batch it with an individually scheduled job in π^E , while following the agent-separation constraint.
- **SwapBatch:** For each batched job in π^E , attempt to swap it with a late job of the same agent from π^L .
- **Delay:** For each individually scheduled job in π^E , consider moving it to the late job set π^L to create space for other jobs.
- **SplitBatch:** For each batch in π^E , attempt to split the batch by moving the longer job to π^L , while keeping the shorter job at its original position in π^E .

The design and selection of these neighborhood moves are guided by our experimental trials. Traditional swap moves, where two arbitrary jobs are exchanged, are found to be ineffective and time-consuming.

The *InsertLate* move is useful when adjustments to completion times allow previously late jobs to be scheduled early. The *BatchLate* move addresses cases where jobs with relatively late due dates can still be batched effectively with jobs that have earlier due dates, allowing better utilization of available time slots. For example, as shown in Fig. 1, the *BatchLate* move enables job J_2^B to move forward and form a batch with job J_1^A . The *SwapBatch* move targets cases where scheduling solely based on due dates results in batches composed of jobs with significantly different processing times. Ideal batching generally occurs between jobs of similar processing lengths, thus reducing total completion time more effectively. Fig. 2 provides an illustration: after swapping jobs J_1^A and J_2^A via *SwapBatch*, the subsequent application of the *InsertLate* move results in all jobs being scheduled early. For the moves *BatchLate* and *SwapBatch*, if the resulting batch contains at least one job completed late, the corresponding neighborhood solution is discarded. This filtering aims to control the quality of generated solutions and improve the overall efficiency of the search. The *Delay* move is tailored for jobs that are scheduled early but have long processing times and are not batched. These jobs can delay all subsequent jobs. Moving them to the late set may free up time for other jobs to be completed early, thereby improving the overall solution. Note that the above moves do not reduce the number of batches. However, having more batches does not always lead to better performance. We introduce *SplitBatch* as a complementary move to allow more extensive structural changes in the solution.

5.2. Simulated annealing algorithm

Our SA implementation is detailed in Algorithm (3) with the following parameters:

- **Initial temperature** (T_0): Controls the initial acceptance probability for worse solutions.
- **Final temperature** (T_f): The stopping criterion for cooling.
- **Cooling rate** (*cooling rate*): Determines the rate at which temperature decreases.
- **Neighbors per iteration** (*neighbors_per_iter*): Number of neighborhood solutions explored at each iteration.
- **Iterations per temperature** (*iters_per_temp*): Number of iterations conducted at each temperature level.
- **Acceptance scaling parameter** (β): Adjusts acceptance probability for worse solutions.

The algorithm begins with an initial solution obtained from the constructive heuristics (*initial_solution*), denoted as the *current* solution. The *best* solution found, π^* , is initially set as the current solution. SA maintains a temperature parameter, T , initially set to T_0 , and gradually decreases it by a factor of the *cooling rate*, continuing until it reaches the final temperature T_f .

At each temperature level, the algorithm performs a fixed number of iterations (*iters_per_temp*). Then, within each iteration, it generates a fixed number of neighboring solutions (*neighbors_per_iter*). Each neighbor is obtained by randomly selecting and applying one of five predefined neighborhood moves. Unlike classical SA algorithms, which typically evaluate and accept a single random neighbor, our approach identifies the best neighbor among the generated candidates based on their objective values. This modification introduces a more structured local search within each iteration. Our experimental results indicate that both *iters_per_temp* and *neighbors_per_iter* lead to more stable convergence and improved performance. For each candidate neighbor π , the algorithm evaluates one of the following objective functions, denoted by $f(\pi)$, depending on the optimization form under consideration:

- For the weighted sum problem: $f(\pi) = \alpha \sum_{i=1}^{n_A} U_i^A + (1-\alpha) \sum_{j=1}^{n_B} U_j^B$.
- For the constrained problem: $f(\pi) = \sum_{i=1}^{n_A} U_i^A$.

For the constrained problem, each candidate solution must satisfy the feasibility condition $\sum_{j=1}^{n_B} U_j^B \leq Q$. If no feasible neighbor is found within the current iteration, the algorithm proceeds to the next iteration without making any changes to the current solution.

To decide whether the current solution should move to the best neighbor found, the algorithm calculates the difference in objectives (Δ_{obj}). If the neighbor improves the current solution ($\Delta_{\text{obj}} < 0$), it is directly accepted. Otherwise, it is accepted with a probability given by $\exp(-\Delta_{\text{obj}}/(\beta \cdot T))$, where the parameter β controls the acceptance of worse solutions. This probabilistic criterion helps avoid local minima and allows broader exploration of the solution space. If the newly accepted solution improves upon the best solution found so far, it replaces the best solution. After completing all iterations at the current temperature, the temperature is reduced by multiplying by the *cooling_rate*.

The process repeats until the temperature falls below the threshold T_f . The SA algorithm returns the objective value of the best solution found upon termination.

5.3. Tabu search algorithm

The implementation of the proposed TS algorithm is shown in Algorithm (4). The key parameters are summarized below:

- **Maximum iterations** (*max_iter*): Limits total algorithm runtime.
- **Minimum tabu tenure** (*min_tenure*): Lower bound on the tabu list length.
- **Maximum tabu tenure** (*max_tenure*): Upper bound on the tabu list length.
- **No-improvement threshold** (*max_no_improve*): Number of iterations allowed without improvement before restarting.

Similar to SA, TS begins with an initial solution obtained from the constructive heuristic (*initial_solution*). The algorithm iteratively generates neighboring solutions by applying the neighborhood moves defined in Section 5.1. In each iteration, the algorithm evaluates all candidate neighbors generated by these moves and selects the best one based on the objective value $f(\pi)$. Different from SA, TS additionally considers the completion time as a secondary criterion to break ties when multiple neighbors have the same objective value. This strategy was found to be effective in our preliminary experiments.

Each candidate solution generated by a neighborhood move is associated with a unique *move signature*, identifying both the type of move and the specific jobs or positions involved. The tabu list, a short-term memory structure, then maintains these move signatures to avoid revisiting recent solutions, effectively diversifying the search. Specifically, for the moves *BatchLate* and *SwapBatch*, the tabu mechanism prevents the same pair of jobs from being swapped again in the short term. For the *InsertLate* move, the insertion position is marked tabu, preventing

Algorithm 3: Simulated Annealing

Input : Job sets $\mathcal{J}^A, \mathcal{J}^B$; initial temperature T_0 ; final temperature T_f ; cooling rate γ ; iterations per temperature I ; number of neighbors per iteration k ; acceptance parameter β ; α, Q (if applicable)

Output : Approximate solution $\pi^* = (\pi^E, \pi^L)$

Initialization: $\pi \leftarrow$ initial solution from Algorithm (1) or (2);
 $\pi^* \leftarrow \pi, T \leftarrow T_0$

```

1 while  $T > T_f$  do
2   for  $i = 1$  to  $I$  do
3     Let  $\mathcal{N}_k(\pi)$  be a set of  $k$  neighbors of  $\pi$ , generated by
       random application of neighborhood moves;
4     Select  $\pi'$  from  $\mathcal{N}_k(\pi)$  such that  $f(\pi')$  is minimized;
5     if constrained problem then
6       if  $\pi'$  is infeasible (i.e.,  $\sum_{j=1}^{n_B} U_j^B > Q$ ) then
7         continue to next iteration;
8       end
9     end
10     $\Delta \leftarrow f(\pi') - f(\pi)$ ;
11    if  $\Delta < 0$  or  $\text{rand}() < \exp(-\Delta/(\beta T))$  then
12       $\pi \leftarrow \pi'$ ;
13      if  $f(\pi) < f(\pi^*)$  then
14         $\pi^* \leftarrow \pi$ ;
15      end
16    end
17  end
18   $T \leftarrow \gamma T$ ;
19 end
```

other jobs from being inserted into the same position. For the *Delay* move, the job that is moved into the late set is prohibited from being delayed again. For the *SplitBatch* move, the selected pair of jobs is prevented from being batched together. The length of the tabu list, known as the *tabu tenure*, dynamically varies between predefined lower and upper bounds (*min_tenure*, *max_tenure*) to balance diversification and intensification in the search process.

The algorithm employs an *aspiration criterion*: even if a move is marked tabu, it can still be accepted if the resulting solution improves upon the best solution found so far. This criterion ensures that the algorithm does not overlook high-quality solutions due to strict tabu constraints. Additionally, we incorporate a restart mechanism controlled by the parameter *max_no_improve*. If no improvement in the best-found solution is observed after a certain number of iterations (*max_no_improve*), the algorithm resets the current solution to the best-known solution and clears the tabu list. This mechanism prevents excessive stagnation in non-improving regions and allows the algorithm to repeatedly explore different areas of the solution space.

The TS algorithm terminates when the maximum number of iterations (*max_iter*) is reached, returning the objective value of the best solution found.

6. Computational experiments

In this section, we conduct a series of computational experiments to evaluate the performance of our proposed heuristic algorithms. The MILP models are solved using Gurobi solver version 10.0.2. The solver is run with a time limit of 1800 s, while all other parameters are kept at their default settings. All algorithms were implemented in Python 3.11 and executed on a Mac Studio (2023) with an Apple M2 Max processor and 32 GB of RAM, running macOS Sonoma 14.0.

Algorithm 4: Tabu Search

Input : Job sets $\mathcal{J}^A, \mathcal{J}^B$; maximum iterations N ; tabu tenure range $[\tau_{\min}, \tau_{\max}]$; no-improvement threshold L ; α, Q (if applicable)

Output : Approximate solution $\pi^* = (\pi^E, \pi^L)$

Initialization: $\pi \leftarrow$ initial solution from Algorithm (1) or (2);
 $\pi^* \leftarrow \pi, f^* \leftarrow f(\pi)$; tabu list $\mathcal{T} \leftarrow \emptyset$,
no-improvement counter $\ell \leftarrow 0$

```

1 Randomly select initial tabu tenure  $\tau \in [\tau_{\min}, \tau_{\max}]$ ;
2 for  $t = 1$  to  $N$  do
3   Generate neighborhood  $\mathcal{N}(\pi)$  by applying all move types to
    $\pi$ ;
4   foreach  $\pi' \in \mathcal{N}(\pi)$  do
5     if constrained problem and  $\sum_{j=1}^{n_B} U_j^B(\pi') > Q$  then
6       continue to next neighbor;
7     end
8     Compute move signature  $s(\pi, \pi')$  corresponding to the
      move;
9     if  $s(\pi, \pi') \in \mathcal{T}$  and  $f(\pi') \geq f^*$  then
10      continue to next neighbor;
11    end
12    Compute completion time  $C(\pi')$ ;
13    Store best admissible neighbor  $\pi_{\text{cand}}$  with minimal
       $(f(\pi'), C(\pi'))$  in lexicographic order;
14  end
15  if no admissible neighbor is found then
16    break;
17  end
18  Update  $\pi \leftarrow \pi_{\text{cand}}$ , add  $s(\pi, \pi_{\text{cand}})$  to  $\mathcal{T}$ ;
19  If  $|\mathcal{T}| > \tau$ , remove oldest entry from  $\mathcal{T}$ ;
20  if  $f(\pi) < f^*$  then
21     $\pi^* \leftarrow \pi, f^* \leftarrow f(\pi), \ell \leftarrow 0$ ;
22  end
23  else
24     $\ell \leftarrow \ell + 1$ ;
25  end
26  if  $\ell \geq L$  then
27    Reset  $\pi \leftarrow \pi^*$ , clear  $\mathcal{T}, \ell \leftarrow 0$ ;
28    Randomly select new  $\tau \in [\tau_{\min}, \tau_{\max}]$ ;
29  end
30 end

```

6.1. Test instances and parameter tuning

The test instances are designed to mimic a ship-loading operation. The two agents correspond to two clients, each associated with a different mineral type to be loaded in parallel by a dual-arm shiploader. Each client submits multiple loading orders (jobs), characterized by their size and contractual delivery deadline. The number of jobs for each agent is set to $n_A = n_B \in \{10, 20, 50\}$, giving a total of $n = n_A + n_B \in \{20, 40, 100\}$. The *processing times* represent the loading durations of individual orders, determined by order size and the shiploader's transfer rate, and are drawn from a discrete uniform distribution on $[1, 50]$. The *due dates* are sampled from a discrete uniform distribution on $[10, \frac{50}{2} \beta n]$, where $\beta \in \{0.8, 1.0, 1.2\}$ controls deadline tightness and thus the operational pressure level. For the problem 1 | p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$, we consider $\alpha \in \{0.5, \frac{1}{3}\}$. Here, $\alpha = 0.5$ reflects equal business priority between the two minerals, while $\alpha = \frac{1}{3}$ models a case where Agent A carries twice the priority of Agent B. For the constrained variant 1 | p -batch; $b = 2$ | $U_i^A : U_j^B \leq Q$, we set $Q = \gamma n_B$ with $\gamma \in \{0.2, 0.4, 0.6\}$. This bound reflects service-level commitments for Agent B, where smaller γ values impose stricter limits on allowable delays.

The entire dataset consists of 60 parameter combinations for the two problems. For each combination, 10 independent instances are generated.

The performance of the proposed metaheuristics is sensitive to parameter settings. We use a grid search to identify effective configurations through preliminary experiments. For SA, we fix the initial and final temperatures at $T_0 = 10000$ and $T_f = 1$, respectively, and test the following parameters:

- Cooling rate (*cooling_rate*): (0.90, 0.95, 0.99)
- Neighbors per iteration (*neighbors_per_iter*): (10, 15, 20)
- Iterations per temperature (*iters_per_temp*): (5, 10, 15)
- Acceptance scaling parameter (β): (1, 0.1, 0.01)

For TS, we evaluate:

- Maximum iterations (*max_iter*): (4000, 6000, 8000)
- Minimum tabu tenure (*min_tenure*): (50, 100, 150)
- Maximum tabu tenure (*max_tenure*): (150, 200, 250)
- No-improvement threshold (*max_no_improve*): (200, 400, 600)

Each parameter combination is tested on 10 randomly generated instances per setting. Based on average performance, the selected configurations are [*cooling_rate*, *neighbors_per_iter*, *iters_per_temp*, β] = [0.9, 10, 10, 0.1] for SA and [*max_iter*, *min_tenure*, *max_tenure*, *max_no_improve*] = [4000, 100, 250, 400] for TS. To control the computational efficiency, we additionally impose a time limit of 90 s as a termination condition for each algorithm run.

6.2. Experimental results

Gurobi is able to optimally solve all small-size instances ($n = 20$) and a subset of medium- ($n = 40$) and large-size ($n = 100$) instances within the time limit. For instances where the global optimum is not reached, we use the best feasible objective value reported by Gurobi as its output. To compare the performance of the proposed approaches, we use the absolute deviation indicator (*dev*), defined as

$$dev = Alg_{sol} - Min_{sol},$$

where Alg_{sol} denotes the objective value obtained by a given algorithm, and Min_{sol} is defined as the best known objective value among all our approaches (i.e., MILP by Gurobi, SA, and TS). We do not apply the common relative percentage measure, since the objective is the number of late jobs, which may be zero, making the ratio undefined. The absolute indicator remains well-defined even when $Min_{sol} = 0$, but its interpretation should be considered relative to the instance size. The MILP solver (Gurobi) is run with a fixed time limit and may therefore return sub-optimal solutions in some instances. For transparency, we also report the number of instances solved to proven optimality by Gurobi (*No. Opt.*). For each setting, we report the average absolute deviation (*Avg., dev*) over the 10 tested instances. We also report *No., Best*, which counts how many times each algorithm achieves the best solution. In addition, we include the maximum absolute deviation (*Max., dev*), which reflects the worst-case (weighted) number of extra late jobs compared to the best-known solution.

Table 2 summarizes the performance of Gurobi, SA, and TS on 1 | p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$. Considering the problem's inherent complexity, Gurobi performs reasonably well within the time limit. Even in $n = 40$ and $n = 100$, where not all instances are solved to optimality, Gurobi achieves the best result in 9 or all 10 tested instances per setting, with *Avg. dev* remaining below 0.05, i.e., corresponding to at most one extra late job than the best solution. SA consistently shows strong performance across all settings. It achieves the best-known solution in 9 or all 10 instances, with *Avg. dev* not exceeding 0.07. In particular, note that SA's performance improves with problem size. For $n = 100$, SA exhibits clear dominance over the other methods, consistently achieving the best-known solution with minimal deviation.

Table 2Performance of the proposed algorithms for problem 1 | p -batch; $b = 2 \mid \alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$.

n	α	β	Gurobi			SA			TS		
			No. Opt	Avg. dev	No. Best	Avg. dev	No. Best	Max. dev	Avg. dev	No. Best	Max. dev
20	1/3	0.8	10	0.00	10	0.03	9	1/3	0.13	7	2/3
		1.0	10	0.00	10	0.03	9	1/3	0.17	7	2/3
		1.2	10	0.00	10	0.03	9	1/3	0.07	8	1/3
	0.5	0.8	10	0.00	10	0.05	9	0.5	0.15	7	0.5
		1.0	10	0.00	10	0.00	10	0	0.05	9	0.5
		1.2	10	0.00	10	0.05	9	0.5	0.05	9	0.5
40	1/3	0.8	7	0.03	9	0.00	10	0	0.17	7	1
		1.0	8	0.00	10	0.07	9	2/3	0.13	7	2/3
		1.2	9	0.00	10	0.00	10	0	0.07	9	2/3
	0.5	0.8	7	0.05	9	0.00	10	0	0.10	9	1
		1.0	7	0.00	10	0.05	9	0.5	0.05	9	0.5
		1.2	9	0.00	10	0.00	10	0	0.05	9	0.5
100	1/3	0.8	3	0.03	9	0.03	9	1/3	0.13	7	2/3
		1.0	7	0.00	10	0.00	10	0	0.07	8	1/3
		1.2	9	0.03	9	0.00	10	0	0.00	10	0
	0.5	0.8	5	0.05	9	0.00	10	0	0.05	9	0.5
		1.0	7	0.00	10	0.00	10	0	0.05	9	0.5
		1.2	9	0.05	9	0.00	10	0	0.05	9	0.5

Even in instances where Gurobi reaches optimality, SA produces near-zero deviations. Based on these consistent results, we conjecture that the feasible (incumbent) solutions returned by Gurobi are near-optimal, even when optimality is not proven. TS also performs competitively, particularly for larger instances, though it shows slightly greater variability. Its *Avg. dev* remains below 0.17. In terms of maximum absolute deviation, TS does not exceed a value of 1, indicating that at most two additional jobs are mis-scheduled compared to the best-known solution. Parameter sensitivity is also evident from the results. When comparing $\alpha = 0.5$ to $\alpha = 1/3$, the performance of all algorithms improves under $\alpha = 0.5$, which is consistent with the reduced complexity from the unweighted objective function. In addition, lower values of β , which correspond to tighter due dates, are associated with higher deviations for Gurobi and TS. In contrast, SA appears more robust to variations in β .

For the constrained problem 1 | p -batch; $b = 2 \mid \sum U_i^A : \sum U_j^B \leq Q$, we observe a similar pattern, as demonstrated in Table 3. Gurobi provides the optimal solution in most instances, when the instance size is $n = 20$ or $n = 40$. Among the three approaches, it consistently delivers the best results across all tested instances. SA also performs very well, closely matching Gurobi's results, with the maximum *Avg. dev* reaching only 0.30, even when $n = 100$. In particular, under less restrictive due date conditions ($\beta = 1.2$), it obtains the best solution across all instances. While TS also shows decent performance, it is slightly outperformed by SA and exhibits relatively lower stability. We can observe that the constrained problem is slightly harder for both metaheuristics than the weighted-sum counterpart when $n = 100$, likely due to the stricter feasibility requirement on Agent B 's lateness.

Moreover, we conducted an efficiency test for the three solution approaches, as reported in Table 4. For each setting, we report their average and maximum running times across the tested instances. Interestingly, the efficiency of Gurobi for solving the constrained problem is noticeably better than for the weighted-sum problem, as reflected by notable differences in computational times, although some instances remain intractable when $n = 40$ and $n = 100$. As Gurobi's branch-and-bound relies heavily on bounding and pruning, we conjecture that the constrained version enables more effective pruning through the hard bound on Agent B 's objective, while the weighted-sum version must evaluate a larger space of trade-offs, which enlarges the search tree. However, the performance of the two metaheuristics shows the opposite trend. In the weighted-sum version, both SA and TS are extremely efficient: all instances are solved in negligible time for $n = 20$ and $n = 40$, with the maximum running time for $n = 100$ remaining below 10.93 s and 14.18 s, respectively. In the constrained version, while still efficient, the average running times of SA and TS rise to

at most 32.73 s and 48.44 s, respectively. A possible explanation is that repeated feasibility checks in the constrained version introduce additional overhead and reduce the flexibility of neighborhood moves. Overall, SA is the more efficient and stable method across both problem versions (see Table 4).

7. A special case with a constant number of batches

In this section, we examine an extension where the number of batches allowed in the schedule is limited to at most k , where k is a given constant. This scenario is realistic and common in real-life applications where forming a batch incurs a non-negligible cost or consumes limited resources. For example, in manufacturing, each batch may require a separate setup, which is time-consuming and costly. In transportation, the number of available containers or delivery vehicles may impose a hard limit on how many batches can be dispatched per day. In such situations, the number of batches does not scale with the number of jobs n . We show that this restricted variant can be solved in polynomial time, and we provide a procedure that applies effectively to both objective forms.

To solve the general problem, a key challenge lies in the lack of consistent structural properties among batched jobs and individually processed jobs. When the number of batches is constant, we circumvent such complexities by preselecting all batches and fixing their sequence. Let $\Delta = \{\sigma^1, \dots, \sigma^{|\Delta|}\}$ be the set of all possible partial schedules. Each σ^r consists of only batches, with their sequence specified and $|\sigma^r| \leq k$, for $r = 1, \dots, |\Delta|$. For ease of exposition, we let $\sigma^r = \{B_1, \dots, B_{|\sigma^r|}\}$, where each batch B_ℓ contains jobs $J_{B_\ell}^A$ and $J_{B_\ell}^B$. The processing time and due date for each batch are given by $p(B_\ell) = \max\{p_{B_\ell}^A, p_{B_\ell}^B\}$ and $d(B_\ell) = \min\{d_{B_\ell}^A, d_{B_\ell}^B\}$, respectively. Then we partition the problem into $|\Delta|$ subproblems $P(\sigma^1), \dots, P(\sigma^{|\Delta|})$, and for each subproblem $P(\sigma^r)$ we aim to determine the optimal solution, if it exists, among all feasible schedules that integrate the predefined batch sequence σ^r . Following Lemma 4.2, we specifically ensure that all batched jobs are completed early in any feasible solutions.

Below we show that each subproblem can be solved in polynomial time. Given $\sigma^r = \{B_1, \dots, B_{|\sigma^r|}\}$, we first check whether the scheduled batches alone can be completed early in the specified sequence. If this is not possible, the subproblem is clearly infeasible, and we proceed to the next one. If feasible, we employ a dynamic programming algorithm to incorporate the remaining jobs into the existing partial schedule σ^r .

Based on Lemma 4.1, we re-index all jobs within $\mathcal{J}^A \cup \mathcal{J}^B \setminus \sigma^r$ according to the EDD rule. Then let $C_r(j, \ell, h, q)$ be the minimum completion time of the last early job in a partial schedule involving the first j EDD jobs and the first ℓ batches in σ^r , while having at most

Table 3Performance of the proposed algorithms for problem 1 | p -batch; $b = 2$ | $\sum U_i^A : \sum U_j^B \leq Q$.

n	γ	β	Gurobi			SA			TS		
			No. Opt	Avg. dev	No. Best	Avg. dev	No. Best	Max. dev	Avg. dev	No. Best	Max. dev
20	0.1	0.8	10	0.00	10	0.10	9	1	0.30	7	1
		1.0	10	0.00	10	0.00	10	0	0.10	9	1
		1.2	10	0.00	10	0.00	10	0	0.10	9	1
	0.2	0.8	10	0.00	10	0.10	9	1	0.10	9	1
		1.0	10	0.00	10	0.00	10	0	0.00	10	0
		1.2	10	0.00	10	0.00	10	0	0.00	10	0
40	0.1	0.8	9	0.00	10	0.20	9	2	0.30	9	3
		1.0	10	0.00	10	0.00	10	0	0.20	8	1
		1.2	10	0.00	10	0.00	10	0	0.10	9	1
	0.2	0.8	9	0.00	10	0.10	9	1	0.10	9	1
		1.0	10	0.00	10	0.00	10	0	0.00	10	0
		1.2	10	0.00	10	0.00	10	0	0.00	10	0
100	0.1	0.8	7	0.00	10	0.20	8	1	0.10	9	1
		1.0	8	0.00	10	0.30	9	3	0.10	9	1
		1.2	9	0.00	10	0.00	10	0	0.10	9	1
	0.2	0.8	7	0.00	10	0.20	8	1	0.50	6	2
		1.0	8	0.00	10	0.10	9	1	0.40	8	3
		1.2	9	0.00	10	0.00	10	0	0.00	10	0

Table 4

Running times (in seconds) of Gurobi, SA, and TS

Left: 1 | p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$; Right: 1 | p -batch; $b = 2$ | $\sum U_i^A : \sum U_j^B \leq Q$.

n	α	β	Gurobi		SA		TS		n	γ	β	Gurobi		SA		TS	
			Average	Max	Average	Max	Average	Max				Average	Max	Average	Max	Average	Max
20	1/3	0.8	6.08	25.69	0.38	0.53	0.03	0.05	20	0.1	0.8	53.57	432.39	0.54	0.73	0.17	0.22
		1.0	4.45	19.15	0.34	0.44	0.02	0.05			1.0	10.09	25.52	0.52	0.67	0.16	0.19
		1.2	3.10	13.22	0.31	0.41	0.02	0.07			1.2	5.05	10.11	0.47	0.58	0.18	0.19
	0.5	0.8	10.49	43.48	0.38	0.58	0.03	0.06		0.2	0.8	18.87	28.12	0.62	0.90	0.19	0.24
		1.0	7.95	26.88	0.34	0.47	0.04	0.13			1.0	10.08	14.41	0.59	0.77	0.26	0.52
		1.2	3.92	14.39	0.31	0.40	0.02	0.08			1.2	1.05	4.11	0.56	0.72	0.29	0.62
40	1/3	0.8	566.18	>1800	1.10	1.53	0.19	0.60	40	0.1	0.8	362.33	>1800	2.01	2.72	0.71	1.01
		1.0	420.12	>1800	0.88	1.67	0.18	0.62			1.0	17.90	109.95	1.89	2.59	1.21	3.28
		1.2	260.15	>1800	0.79	1.35	0.15	0.53			1.2	15.14	56.21	1.89	2.94	1.86	2.81
	0.5	0.8	571.16	>1800	1.31	2.27	0.23	0.72		0.2	0.8	238.87	>1800	2.50	2.76	3.05	4.21
		1.0	560.34	>1800	0.98	1.94	0.26	0.94			1.0	16.27	48.29	2.72	4.27	3.13	5.55
		1.2	248.89	>1800	0.83	1.65	0.15	0.47			1.2	10.15	42.34	2.59	3.20	3.60	4.43
100	1/3	0.8	1276.52	>1800	6.02	7.79	6.4	11.14	100	0.1	0.8	565.23	>1800	18.56	22.51	25.97	34.00
		1.0	729.66	>1800	4.30	7.32	3.35	14.18			1.0	378.7	>1800	19.62	26.00	24.40	37.46
		1.2	293.18	>1800	3.40	3.97	3.75	9.14			1.2	197.37	>1800	19.14	27.03	29.03	37.70
	0.5	0.8	969.69	>1800	7.53	10.93	6.77	10.37		0.2	0.8	550.81	>1800	28.64	35.36	44.99	57.85
		1.0	569.30	>1800	4.31	7.44	6.84	18.13			1.0	380.13	>1800	32.73	39.28	48.44	56.04
		1.2	295.49	>1800	3.51	3.93	5.12	9.54			1.2	188.64	>1800	32.10	36.85	46.38	57.67

h and q jobs of Agents A and B being late, respectively. At each step, the recursion schedules either the next EDD job (as early or late) or the next batch. Again, a batch is scheduled only if both jobs in it are completed early. All possible cases are summarized in the following Recursion (19):

$$C_r(j, \ell, h, q) = \min \begin{cases} C_r(j-1, \ell, h, q) + p_j & \text{if } C_r(j-1, \ell, h, q) + p_j \leq d_j \\ C_r(j-1, \ell, h-1, q) & \text{if } J_j \in \mathcal{J}^A \\ C_r(j-1, \ell, h, q-1) & \text{if } J_j \in \mathcal{J}^B \\ C_r(j, \ell-1, h, q) + p(B_\ell) & \text{if } C_r(j-1, \ell-1, h, q) + p(B_\ell) \leq d(B_\ell) \end{cases} \quad (19)$$

with initial conditions:

$$C_r(j, \ell, h, q) = \begin{cases} 0 & \text{if } j = \ell = h = q = 0 \\ +\infty & \text{otherwise} \end{cases} \quad (20)$$

Lemma 7.1. If $C_r(j, \ell, h, q)$ is finite, it represents the minimum completion time of the last early job for all feasible partial schedules involving jobs $J_1, \dots, J_j$ and batches $B_1, \dots, B_\ell$, with at most h and q late jobs from Agents A and B , respectively. If it is infinite, no feasible schedule exists under these conditions.

Proof. We refer the reader to Agnetis et al. (2004) for the proof applied to the classical two-agent problem without batching. $\square$

Theorem 7.2. For 1| p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$, an optimal solution to the subproblem $P(\sigma^r)$ can be obtained by evaluating

$$C^* = \inf_{\substack{h \in \{0, \dots, n_A - |\sigma^r|\} \\ q \in \{0, \dots, n_B - |\sigma^r|\}}} \{ \alpha h + (1 - \alpha) q : C(n - 2|\sigma^r|, |\sigma^r|, h, q) < +\infty \}.$$

For 1| p -batch; $b = 2$ | $\sum U_i^A : \sum U_j^B \leq Q$, an optimal solution to the subproblem $P(\sigma^r)$ can be obtained by evaluating

$$C^* = \inf_{h \in \{0, \dots, n_A - |\sigma^r|\}} \{ h : C(n - 2|\sigma^r|, |\sigma^r|, h, Q) < +\infty \}.$$

Upon solving all subproblems, the global optimal solution is the one yielding the lowest objective value.

Theorem 7.3. When the schedule is restricted to contain at most k batches, 1| p -batch; $b = 2$ | $\alpha \sum U_i^A + (1 - \alpha) \sum U_j^B$ and 1| p -batch; $b = 2$ | $\sum U_j^A : \sum U_j^B \leq Q$ can be solved in $\mathcal{O}(n \cdot n_A^{k+1} \cdot n_B^{k+1})$ time.

Proof. The initial sorting of jobs takes $\mathcal{O}(n \log(n))$ time. Since the schedule includes at most k batches, we select at most k jobs from $\mathcal{J}^A$ and $\mathcal{J}^B$, respectively. Accordingly, there are $|\Delta| = \mathcal{O}(n_A^k n_B^k)$ possible batch selections and sequences. For each subproblem $P(\sigma^r)$, the

feasibility check requires $\mathcal{O}(n)$ time. While the computation of each $C_r(j, \ell, h, q)$ takes constant time, the state variable j is bounded by n , ℓ is bounded by k , and h and q can take n_A and n_B values, respectively. The final evaluation of C^* involves checking all pairs of h and q values. Therefore, the overall running time is $\mathcal{O}(n \cdot n_A^{k+1} \cdot n_B^{k+1})$. $\square$

8. Concluding remarks

In this paper, we study two-agent scheduling problems on a single parallel-batching machine, where each batch can contain at most one job from each agent. The objective is to minimize the number of late jobs, under two optimization variants. Both problems are proven to be strongly $\mathcal{NP}$ -hard through a reduction from the 3-PARTITION problem. We formulate position-based MILP models to obtain optimal solutions. We then analyze structural properties of optimal schedules and design constructive heuristics tailored to each problem. Building on these heuristics, we develop two metaheuristic algorithms: SA and TS. Their parameter configurations, search mechanisms, and neighborhood structures are customized based on preliminary tests. We conduct comprehensive computational experiments to evaluate these approaches. Gurobi shows strong performance, delivering optimal or best-known solutions in nearly all tested instances. However, its runtime becomes impractical for instances with more than 20 jobs per agent. Both metaheuristics perform effectively, with SA showing superior consistency in both solution quality and efficiency. Across both problem variants, SA achieves average absolute deviations from the optimal or best-known solutions below 0.07 and 0.30, respectively, with average running times capped at 7.53 s and 32.73 s, even for instances with 100 jobs. This suggests that SA is well suited for time-sensitive applications that require rapid rescheduling or real-time decision-making. In addition, the results confirm the complementary strengths of the two metaheuristics. The stable performance of SA highlights its robustness in rugged search spaces created by batching rules, agent separation, and lateness-based objectives. Similar strength can also be expected in other scheduling contexts, such as multi-objective scheduling and job-shop problems with complex constraints. TS is more sensitive to parameter tuning, but it can outperform SA when well tuned. Its systematic intensification makes it a promising option in settings with more structured job distributions or larger instances, such as flow-shop or machine scheduling problems with clear bottlenecks.

Future research may explore alternative objectives such as total late work or total tardiness, and extend the model to other machine environments. Multi-machine settings with machine-specific batching restrictions or different processing speeds are particularly interesting. Current research in this intersection is limited due to its intrinsic complexity. Uncertainty in parallel batching is also prevalent and practically important. For instance, a batching machine may be subject to unexpected breakdowns, introducing idle time. One could develop robust or stochastic scheduling algorithms that incorporate contingency buffers or dynamic rescheduling policies to adapt when disruptions occur. Another promising direction is to study more general batch processing settings, including expanded batch capacities or different batch rules. Additionally, applying parameterized complexity theory offers a structured way to further analyze computational complexity (Garey & Johnson, 1979). By treating one or more natural parameters as bounded (constant), it is interesting to investigate fixed-parameter tractability results. The extension discussed in Section 7, where the number of batches is treated as a parameter, illustrates how such an approach can yield tractable cases for otherwise intractable problems.

CRedit authorship contribution statement

Renjie Yu: Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization. **Daniel Oron:** Writing – original draft, Supervision, Methodology, Investigation, Formal analysis, Conceptualization.

Data availability

No data was used for the research described in the article.

References

- Agnetis, A., Mirchandani, P. B., Pacciarelli, D., & Pacifici, A. (2004). Scheduling problems with two competing agents. *Operations Research*, 52(2), 229–242.
- Baker, K. R., & Cole Smith, J. (2003). A multiple-criterion model for machine scheduling. *Journal of Scheduling*, 6(1), 7–16.
- Beldar, P., Moghtader, M., Giret, A., & Ansariipoor, A. H. (2022). Non-identical parallel machines batch processing problem with release dates, due dates and variable maintenance activity to minimize total tardiness. *Computers & Industrial Engineering*, 168, Article 108135.
- Brucker, P., Gladky, A., Hoogeveen, H., Kovalyov, M. Y., Potts, C. N., Tautenhahn, T., & Van De Velde, S. L. (1998). Scheduling a batching machine. *Journal of Scheduling*, 1(1), 31–54.
- Chen, C., Fathi, M., Khakifirooz, M., & Wu, K. (2022). Hybrid tabu search algorithm for unrelated parallel machine scheduling in semiconductor fabs with setup times, job release, and expired times. *Computers & Industrial Engineering*, 165, Article 107915.
- Fan, B. Q., Cheng, T. C. E., Li, S. S., & Feng, Q. (2012). Bounded parallel-batching scheduling with two competing agents. *Journal of Scheduling*, 16(3), 261–271.
- Feng, Q., Shang, W.-P., Jiao, C.-W., & Li, W.-J. (2020). Two-agent scheduling on a bounded parallel-batching machine with makespan and maximum lateness objectives. *Journal of the Operations Research Society of China*, 8(1), 189–196.
- Fontes, D. B., Homayouni, S. M., & Gonçalves, J. F. (2023). A hybrid particle swarm optimization and simulated annealing algorithm for the job shop scheduling problem with transport resources. *European Journal of Operational Research*, 306(3), 1140–1157.
- Fowler, J. W., & Mönnch, L. (2022). A survey of scheduling with parallel batch (p-batch) processing. *European Journal of Operational Research*, 298(1), 1–24.
- Gao, Y., Yuan, J., Ng, C. T., & Cheng, T. (2019). A further study on two-agent parallel-batch scheduling with release dates and deteriorating jobs to minimize the makespan. *European Journal of Operational Research*, 273(1), 74–81.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. New York, NY: Freeman.
- Graham, R. L., Lawler, E. L., Lenstra, J. K., & Kan, A. R. (1979). Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics*, 5, 287–326.
- Kovalyov, M. Y., & Šešok, D. (2019). Two-agent scheduling with deteriorating jobs on a single parallel-batching machine: refining computational complexity. *Journal of Scheduling*, 22(5), 603–606.
- Liu, M., Yang, X., Chu, F., Zhang, J., & Chu, C. (2020). Energy-oriented bi-objective optimization for the tempered glass scheduling. *Omega*, 90, Article 101995.
- Ou, J., Lu, L., & Zhong, X. (2023). Parallel-batch scheduling with rejection: Structural properties and approximation algorithms. *European Journal of Operational Research*, 310(3), 1017–1032.
- Pei, J., Wei, J., Liao, B., Liu, X., & Pardalos, P. M. (2019). Two-agent scheduling on bounded parallel-batching machines with an aging effect of job-position-dependent. *Annals of Operations Research*, 294(1–2), 191–223.
- Shafiee, M., Amiri-Aref, M., & Klibi, W. (2025). The integration of shared renewable resources considering setup times for the parallel machine scheduling problem. *Computers & Industrial Engineering*, 200, Article 110828.
- Shahvari, O., & Logendran, R. (2017). An enhanced tabu search algorithm to minimize a bi-criteria objective in batching and scheduling problems on unrelated-parallel machines with desired lower bounds on batch sizes. *Computers & Operations Research*, 77, 154–176.
- Tang, L., Zhao, X., Liu, J., & Leung, J. Y.-T. (2017). Competitive two-agent scheduling with deteriorating jobs on a single parallel-batching machine. *European Journal of Operational Research*, 263(2), 401–411.
- Wang, J. Q., Fan, G. Q., Zhang, Y., Zhang, C. W., & Leung, J. Y. T. (2017). Two-agent scheduling on a single parallel-batching machine with equal processing time and non-identical job sizes. *European Journal of Operational Research*, 258(2), 478–490.
- Yang, F., Davari, M., Wei, W., Hermans, B., & Leus, R. (2022). Scheduling a single parallel-batching machine with non-identical job sizes and incompatible job families. *European Journal of Operational Research*, 303(2), 602–615.
- Zhang, C. L., Wang, J. Q., & Zhang, C. W. (2019). Two-agent scheduling on a single parallel-batching machine to minimize the weighted sum of the agents' makespans. *Journal of Ambient Intelligence and Humanized Computing*, 10(3), 999–1007.
- Zheng, S., Xie, N., Wu, Q., & Liu, C. (2025). Novel mathematical formulations for parallel-batching processing machine scheduling problems. *Computers & Operations Research*, 173, Article 106859.